

SOLID Principles — Practice Tasks

From Brute Force Design → SOLID Design

Two hands-on assignments for practicing SOLID principles independently.

How to Approach These Tasks

You have already studied the Bird System as a guided example. These two tasks are intentionally different domains. Your job is to discover the design problems yourself and improve the design step by step.

```
STEP 1 → Build the brute-force version
STEP 2 → Identify the problems
STEP 3 → Apply SRP
STEP 4 → Apply OCP
STEP 5 → Look for LSP violations
STEP 6 → Apply ISP where required
STEP 7 → Apply DIP
STEP 8 → Use Dependency Injection
STEP 9 → Draw the final class diagram
```

Do not start by creating interfaces everywhere. For each principle, first identify the problem that the principle is helping you solve.

TASK 1 — Payment Processing System

Objective

Design a Payment Processing System and apply SOLID principles to improve the initial design.

Scenario

You are building a payment system for an e-commerce application.

- Credit Card
- UPI
- Cash

The system should process payments based on the payment type selected by the customer.

Initial Requirements

Initially, all payment-related logic is handled inside a single Payment class.

```
processPayment()
refundPayment()
validatePayment()
```

The system uses conditions similar to:

```
if payment type is Credit Card
    process Credit Card
else if payment type is UPI
    process UPI
```

```
else if payment type is Cash
    process Cash
```

New Requirements

- Add PayPal.
- Add Wallet payments.
- Support refunds.
- Support different payment validation mechanisms.
- Some payment methods do not support refunds.
- Different payment providers can process the same payment method.
- Payment processing logic should be easily replaceable.

Your Task

Start with the brute-force design. Then redesign the system by applying SOLID principles.

During your redesign, answer:

- Where is SRP violated?
- Where is OCP violated?
- Can every payment type be substituted safely?
- Where can ISP be useful?
- Where is DIP violated?
- Where can Dependency Injection be used?

Expected Outcome

```
CreditCard
UPI
Cash
PayPal
Wallet
```

Adding or changing a payment method should not require continuously modifying one large Payment class.

Do not directly use design patterns unless you can explain why they are needed.

TASK 2 — Notification System

Objective

Design a Notification System and apply SOLID principles to make it extensible and maintainable.

Scenario

You are building a notification system for an application.

- Email
- SMS
- Push Notification

A single NotificationService is initially responsible for sending all notifications.

Initial Design

```
sendNotification(type, message)

    if type == EMAIL
        send email
    else if type == SMS
        send SMS
    else if type == PUSH
        send push notification
```

New Requirements

- Add WhatsApp notifications.
- Add Slack notifications.
- Different notification channels have different configuration requirements.
- Some channels support attachments.
- Some channels do not support attachments.
- Some notifications can be retried.
- The application should be able to switch notification providers.
- Adding a new notification channel should not require modifying existing notification implementations.

Your Task

Start by implementing the brute-force design. Then gradually redesign it using SOLID principles.

Answer these questions during your redesign:

- What responsibilities are currently inside NotificationService?
- Which responsibilities should be separated?
- What happens when WhatsApp is added?
- Is the current design open for extension?
- Can every notification type have the same contract?
- Should every notification support attachments?
- Should every notification support retry?
- What should NotificationService depend on?
- Who should create the concrete notification implementation?

Expected Outcome

```
Email
SMS
Push
WhatsApp
Slack
```

Adding a new notification channel should not turn NotificationService into a large collection of if/else statements.

Student Submission

- Brute-force implementation.
- List of problems found in the brute-force design.
- Step-by-step redesign showing where each SOLID principle is applied.
- Final source code.

- Final UML/class diagram.
- A short explanation of why each major design decision was made.

Challenge Rule

Do not try to force every SOLID principle into every part of the system.

If a principle is not relevant to a particular design decision, explain why.

The objective is not to create code that merely looks SOLID. The objective is to learn how to look at a design, identify the actual problem, and choose the appropriate abstraction.

Final Goal

```
Requirement
↓
Brute-force design
↓
Identify the design problem
↓
Choose the relevant principle
↓
Change the design
↓
New requirement
↓
Repeat
```

By the end, you should be able to look at an unfamiliar system and reason about its design instead of memorizing SOLID definitions.